Credit Scoring
For Better decision Making

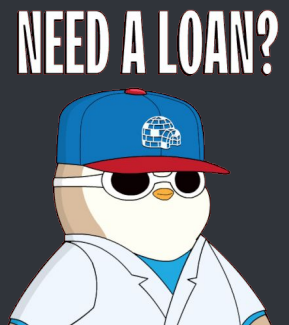
Done By:
Raed Alouani
Amira Farrah

- GiveMeSomeCredit Dataset
- 5 ML Models Compared
- AUC-ROC = 0.9532
- Streamlit Deployment

Imagine walking into a bank and asking for a loan…

Within seconds, the system already knows your risk level — before a human even reviews your file

BUT



How do we make sure that decision is fast, fair, and accurate… when real-world data is messy, incomplete, and highly imbalanced?

This is exactly the challenge we tackled in our project:

Predicting credit default risk using machine learning

Context

- Banks need to assess borrower risk before granting credit.
- Manual evaluation is slow, inconsistent and error-prone.
- An automated ML model provides fast, objective decisions.

Problem

- Predict whether a borrower will default within 2 years.
- Target: SeriousDlqin2yrs (0 = No default, 1 = Default)
- Severe class imbalance: only 6.7% of defaults.

SOLUTION

EDA And Preprocessing

SMOTE (class balancing)

4 Baseline Models

GridSearchCV Optimization

Streamlit Deployment

1 Dataset & Exploratory Analysis

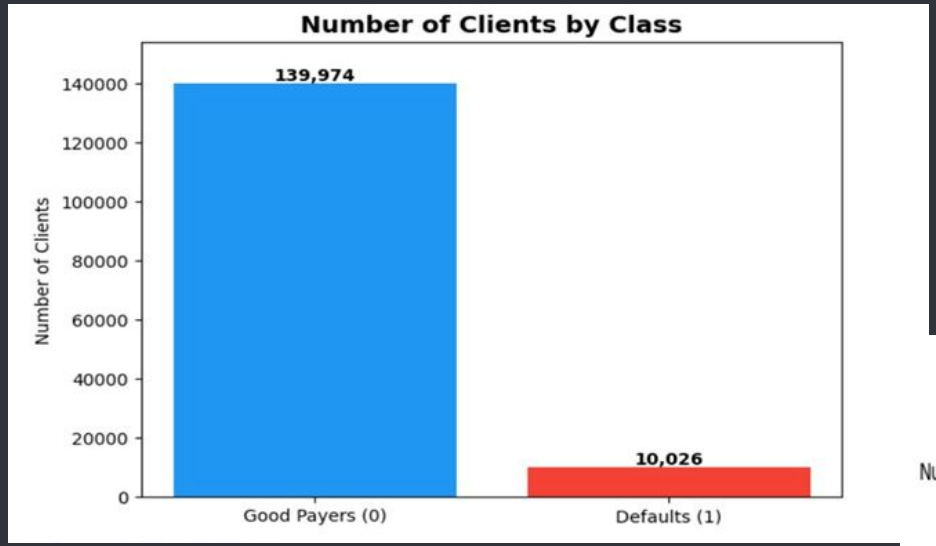
df.head()

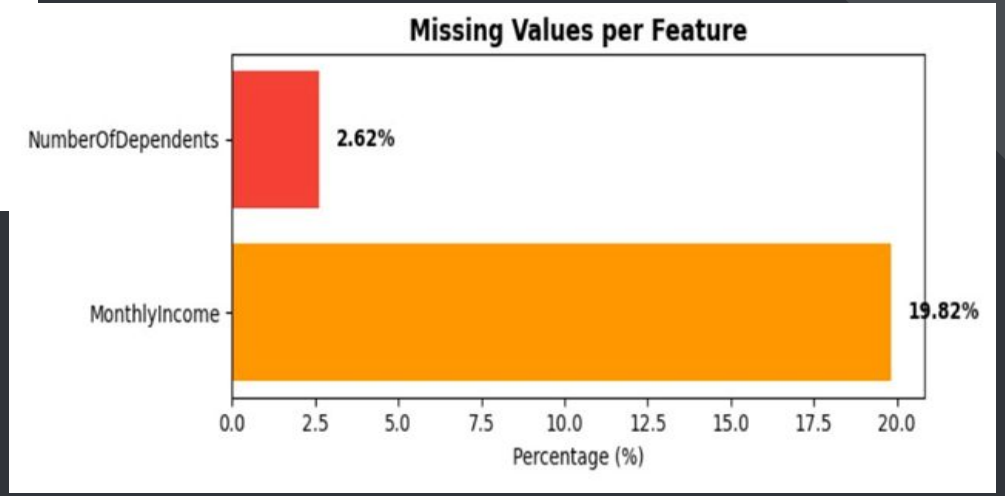
	SeriousDlqin2yrs	RevolvingUtilizationOfUnsecuredLines	age	NumberOfTime30-59DaysPastDueNotWorse	DebtRatio	MonthlyIncome	NumberOfOpenCreditLinesAndLoans
1	1	0.766127	45	2	0.802982	9120.0	13
2	0	0.957151	40	0	0.121876	2600.0	4
3	0	0.658180	38	1	0.085113	3042.0	2
4	0	0.233810	30	0	0.036050	3300.0	5
5	0	0.907239	49	1	0.024926	63588.0	7

NumberOfTimes90DaysLate	NumberRealEstateLoansOrLines	NumberOfTime60-89DaysPastDueNotWorse	NumberOfDependents
0	6	0	2.0
0	0	0	1.0
1	0	0	0.0
0	0	0	0.0
0	1	0	0.0

Dataset Variables Categorization

Category	Variable Name	Description
1. The Target (The Goal)	`SeriousDlqin2yrs`	**1** if the customer had a 90+ days delinquency (Default), **0** if they are a good payer.
2. Customer Profile	`age`	Age of the borrower in years.
2. Customer Profile	`MonthlyIncome`	Monthly gross income (Note: Contains missing values).
2. Customer Profile	`NumberOfDependents`	Number of family dependents (children, spouse, etc.).
3. Credit History	`DebtRatio`	Monthly debt payments divided by monthly gross income.
3. Credit History	`NumberOfOpenCreditLinesAndLoans`	Total number of open loans (car, credit cards, etc.).
3. Credit History	`NumberRealEstateLoansOrLines`	Number of mortgage and real estate loans.
3. Credit History	`RevolvingUtilization`	Percentage of total credit limit currently being used.
4. Red Flags (Past Behavior)	`NumberOfTime30-59DaysPastDue`	Count of 1-2 month payment delays in the last 2 years.
4. Red Flags (Past Behavior)	`NumberOfTime60-89DaysPastDue`	Count of 2-3 month payment delays in the last 2 years.
4. Red Flags (Past Behavior)	`NumberOfTimes90DaysLate`	Count of 3+ month payment delays (Highest risk indicator).





150,000	10	14 : 1	19.8%
Clients	Features	Imbalance Ratio	Missing (Income)

2 Data Preprocessing

01 🔧 Median Imputation

MonthlyIncome ⟶ 19.82% missing

NumberOfDependents ⟶ 2.62% missing

Replaced by column median

⟶ Robust to outliers vs mean imputation

3.2 : Imputation by the median

```
for col in ['MonthlyIncome', 'NumberOfDependents']:
    median_val = data[col].median()
    data[col].fillna(median_val, inplace=True)
    #The role of fillna is to replace missing or null values (NaN) with a specific "filler" value
    #so that our dataset is complete and ready for mathematical calculations.

    print(f" {col} : {data[col].isnull().sum()} missing values (médiane = {median_val:.0f})")
```

```
MonthlyIncome : 0 missing values (médiane = 5400)
NumberOfDependents : 0 missing values (médiane = 0)
```

02 ✂️ Outlier Capping (Winsorization)

6 features treated for outliers

Cap at 99th percentile

Preserves distribution shape

→ **Reduces extreme value impact**

```python
# List of columns to treat for outliers
outlier_cols = ['RevolvingUtilizationOfUnsecuredLines', 'DebtRatio', 'MonthlyIncome',
                'NumberOfTimes90DaysLate', 'NumberOfTime30-59DaysPastDueNotWorse',
                'NumberOfTime60-89DaysPastDueNotWorse']

# Apply 99th percentile capping (Winsorization)
for col in outlier_cols:
    cap = data[col].quantile(0.99)
    data[col] = data[col].clip(upper=cap)

# Verification prints
print("\n Outlier capping completed.")
print(f"\nFinal Shape: {data.shape}")
print(f"Missing Values: {data.isnull().sum().sum()}")
```

```
Outlier capping completed.

Final Shape: (150000, 11)
Missing Values: 0
```

03 ⚖️ SMOTE — Class Balancing

Before: 111,979 vs 8,021 (14:1)

After: 111,979 vs 111,979 (1:1)

Synthetic minority oversampling

→ **Prevents model bias toward class 0**

3.6 : Train/Test split

```
smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)
print(f"\nAfter SMOTE :")
print(f"  Train : {X_train_sm.shape[0]:,} examples ({pd.Series(y_train_sm).value_counts().to_dict()})")
```

```
After SMOTE :
  Train : 223,958 examples ({0: 111979, 1: 111979})
```

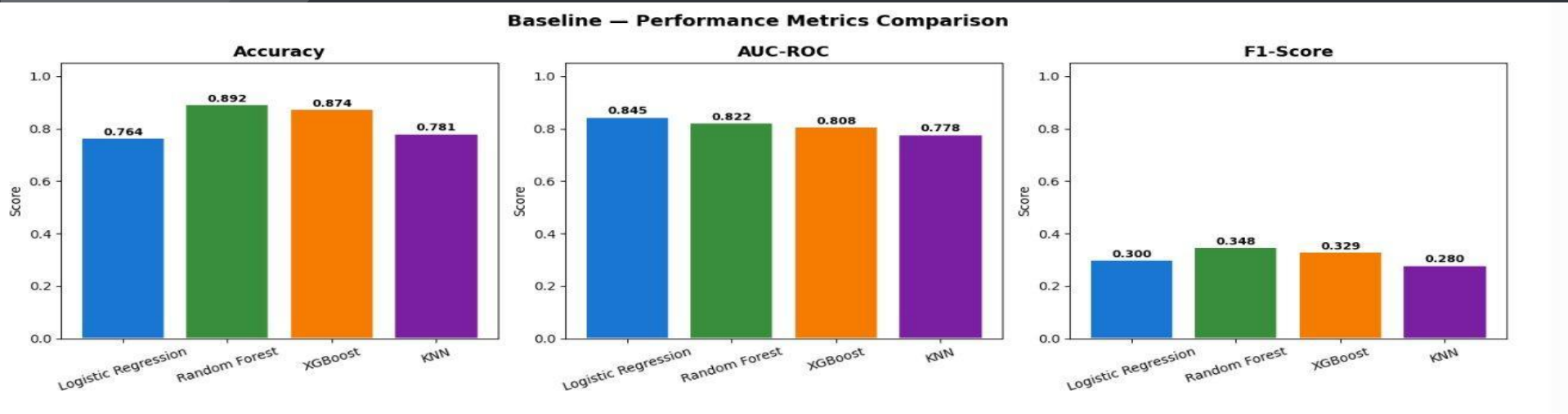
3 Model Training & Comparison

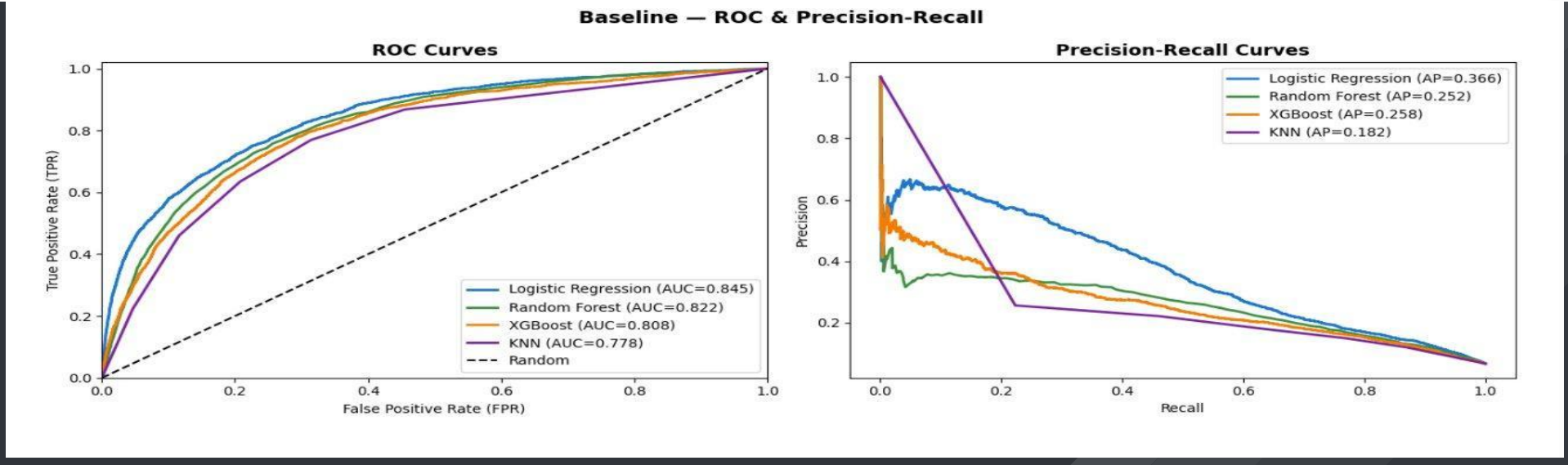
MODELS

Logistic Regression	Linear baseline · Fast · Interpretable
Random Forest	Ensemble · Handles non-linearity
XGBoost	Gradient boosting · High performance
KNN	Distance-based · Non-parametric

Evaluation Metrics

AUC-ROC	Primary metric — handles imbalance
F1-Score	Balance precision & recall
Accuracy	Overall correctness
Avg Precision	Precision-Recall tradeoff


Baseline — Performance Metrics Comparison
Accuracy
AUC-ROC
F1-Score
Score
0.764
0.892
0.874
0.781
0.845
0.822
0.808
0.778
0.300
0.348
0.329
0.280
Logistic Regression
Random Forest
XGBoost
KNN

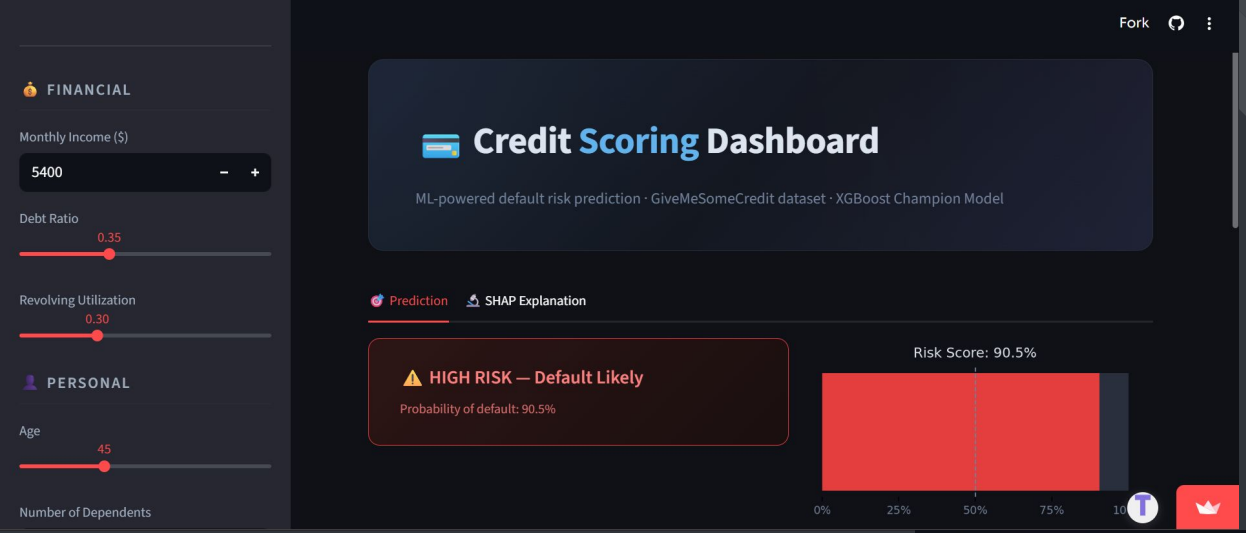

Baseline — ROC & Precision-Recall
ROC Curves
True Positive Rate (TPR)
False Positive Rate (FPR)
Logistic Regression (AUC=0.845)
Random Forest (AUC=0.822)
XGBoost (AUC=0.808)
KNN (AUC=0.778)
Random
Precision-Recall Curves
Precision
Recall
Logistic Regression (AP=0.366)
Random Forest (AP=0.252)
XGBoost (AP=0.258)
KNN (AP=0.182)

AFTER MODEL OPTIMIZATION

Model	AUC-ROC ↑	F1-Score	Accuracy	Avg Precision
Logistic Regression ★	**0.8451**	0.3002	0.7641	0.3656
Random Forest	0.8215	0.3484	0.8924	0.2524
XGBoost	0.8083	0.3292	0.8742	0.2658
KNN	0.7782	0.2797	0.7813	0.2109
XGBoost (Optimized) 🏆	**0.9532**			

🏆 **XGBoost Optimized → AUC = 0.9532**

DASHBOARD



DASHBOARD LINK

We didn’t just predict default risk,
We taught a system
how to make a decision under uncertainty

Because in real life, **data** is **never perfect**…
and **decisions are never easy**

And what we built…
is a system that can handle both

THANK YOU